# The first lower bound improvement on the unrestricted circuit complexity of 3-SAT.

A step towards P $\neq$ NP.

Theodoros Yianni[*,†] Sahibzada Allahyar[†]

Contact: smaa4@cantab.ac.uk

**Abstract**

To our knowledge, this research line gives the first fixed-factor improvement over the elementary $n-1$ circuit lower bound for 3-SAT in the binary encoding and unrestricted two-input gate model specified here. This paper raises the bound to $(17/7-o(1))n$, where $n$ counts the bits describing the formula. Every circuit using arbitrary two-input Boolean gates must meet this bound, even with unrestricted depth and sharing of intermediate results. This priority claim concerns the research line: earlier companion manuscripts establish coefficients 7/3 and 12/5, and this paper raises the coefficient to $17/7 \approx 2.43$. The proof embeds difficult Boolean tests in variable identifiers while retaining almost all the original input bits, then bounds the cost of combining these tests using a two-level gate-elimination measure. The contribution is an unconditional quantitative lower bound for satisfiability itself, measured against the full formula description. It remains a linear bound and does not resolve P versus NP.

## 1 Introduction

**To our knowledge, this research line gives the first fixed-factor improvement over the elementary circuit lower bound for 3-SAT in the encoding and unrestricted binary circuit model studied here.** This paper raises the leading coefficient from the elementary value of one to $17/7 \approx 2.43$ gates per input bit. The input is a specified binary formula encoding, and the circuit may use any two-input Boolean gate, unrestricted depth, and arbitrary sharing. We have located no earlier published fixed-factor improvement over the elementary $n-1$ bound for this exact setting. The first-improvement claim therefore remains qualified by the encoding, circuit model and literature located; within our research line, the present theorem strengthens the earlier 7/3 and 12/5 bounds [25].

**Why progress at this scale matters.** The difficulty of unrestricted circuit lower bounds is a longstanding theme in accounts by researchers who helped develop computational complexity. Sanjeev Arora, a Princeton professor, two-time Gödel Prize recipient, ACM Prize in Computing recipient and member of the US National Academy of Sciences [27], and Boaz Barak, a Harvard professor, ACM Fellow and Simons Investigator [28], wrote in their complexity textbook: "Progress on general circuits has been almost nonexistent" [21]. Lance Fortnow, an ACM Fellow, former chair of ACM SIGACT and former founding dean of Illinois Tech's College of Computing [29], wrote in 2003 that "circuit lower bounds hit a brick wall" [22]. Their expertise and textbooks, research and community leadership make these assessments useful historical context for why even improvements to linear lower bounds receive attention.

Oded Goldreich, a Weizmann researcher awarded the Israel Prize for contributions to cryptography and complexity theory [30], highlighted in 2016 that surpassing the $3n - o(n)$ bound for explicit functions had taken more than thirty years, even for a small increase [26]. Jiatu Li and Tianqi Yang, authors of the STOC 2022 result giving $3.1n - o(n)$ for an explicit function, describe circuit lower bounds as "important but extremely hard" [6]. These observations concern

[*]Department of Computer Science, RHUL

[†]These authors contributed equally and share first authorship.

the broader lower-bound problem; the numerical records are for other explicit functions. They explain the historical setting of our question, rather than establish the priority or correctness of this paper.

3-SAT is a foundational NP-complete problem: given a conjunction of clauses containing three literals each, decide whether some assignment makes every clause true [16]. Its central role motivates asking what circuitry is unavoidable for deciding satisfiability from the formula description.

**The result in plain language.** Our input is the entire formula written in binary, including the variable numbers and signs. Let $n$ denote its length in bits. A circuit may perform any Boolean operation on two values in one gate, use any depth, and share an intermediate result across arbitrarily many clauses. For the family defined in Section 2, we prove

$$C_{B_2}(F_w) \geq \frac{17}{7}n - O\left(\frac{n \log\log n}{\log n}\right).$$

For every $\varepsilon > 0$ and all sufficiently large lengths in this family, fewer than $(17/7 - \varepsilon)n$ gates cannot suffice. This concerns decision circuits for all encoded inputs of a given length, rather than the cost of solving one particular formula.

**What makes this a contribution.** Dependence on every input bit gives only $n-1$ gates. Our theorem forces a larger leading coefficient even when the circuit can exploit arbitrary Boolean operations and sharing. The difficulty is preserving that coefficient when moving from a constructed hard function to a SAT description. A polynomial-time reduction alone does not do this. Our embedding retains an asymptotically full fraction of the description bits, making the bound meaningful on SAT's original input scale.

## The historical record and the SAT transfer problem

Gate elimination has been studied since at least Schnorr's work in 1974 [1]. Stockmeyer and Paul obtained leading coefficients of 5/2 for particular explicit functions [2,3], and Blum obtained 3 in 1984 [4]. That record lasted over thirty years: Find, Golovnev, Hirsch and Kulikov surpassed it in 2016 [5], followed by Li and Yang's $3.1n-o(n)$ result [6]. Mishra and Santhanam subsequently posted a $3.5n - o(n)$ bound in 2026 [7]. These results concern other explicit functions; they are not corresponding bounds for the SAT encoding studied here.

If a reduction encodes a $t$-bit hard function in a formula with $t^2$ description bits, a lower bound proportional to $t$ is only proportional to the square root of the new length, even before charging for the reduction. Our theorem addresses this transfer problem. In the published literature we have located, we know of no earlier fixed-factor improvement over $n - 1$ for the exact encoding and unrestricted binary basis of Section 2. This is an encoding-specific priority statement, not a claim that 3-SAT has seen no lower-bound progress in half a century. Earlier manuscripts in the same research line give the 7/3 and 12/5 stages; the present paper develops the 17/7 argument [25].

## Algorithmic upper bounds and circuit lower bounds

The algorithmic history provides motivation but measures a different kind of progress. Writing $v$ for the number of propositional variables, exhaustive search examines $2^v$ assignments. Schöning's 1999 randomized algorithm achieves $O^*((4/3)^v)$ time [23]. Hertli's 2011 analysis of PPSZ gives $O(1.308^v)$ time for general 3-SAT [24]. Here $O^*$ suppresses polynomial factors. Schöning was not the first to beat exhaustive search: his paper explicitly discusses earlier algorithms doing so.

An upper bound exhibits one procedure that succeeds; a circuit lower bound rules out *every* circuit below a size threshold in the chosen model. Both are valuable, but changing $2^v$ to $c^v$

with $c < 2$ gives an exponential speedup, whereas changing $n$ to $(17/7)n$ gives a constant-factor strengthening of a lower bound. Moreover, $v$ and $n$ are different parameters: here $v = m = 2^w - 1$ and $n = 3m(w + 1)$. The two lines of work measure complementary aspects of SAT complexity: the efficiency of known algorithms and the amount of circuitry that no implementation can avoid.

**Relation to P versus NP.** If P equals NP, this language has polynomial-size circuits. Ruling out every polynomial size would separate P from NP. The present theorem remains compatible with linear-size circuits of a larger constant. Its contribution is a stronger unconditional quantitative statement about satisfiability and an embedding that makes lower-bound techniques effective at the scale of its binary description.

### The proof idea and the new ingredients

The variable identifiers do two jobs: they name propositional variables, and their free bits carry Boolean tests. Choose a set of identifiers and use a small fraction of the clauses to force their variables false. Make the remaining clauses positive and leave most identifier bits free. The formula is satisfiable exactly when each remaining clause names at least one variable outside the forced set. A common assignment witnesses this: set all unforced variables true.

The argument has three parts:

1. **Preserve the input scale.** The identifier embedding leaves $n - o(n)$ free bits. Restriction accounting also credits the fixed bits, so the final bound is measured against the original $n$.

2. **Choose tests that resist simplification.** Strong two-dependence, due to Lachish and Raz [9], ensures that the four assignments to two inputs leave four distinct residual functions even after many other inputs are fixed. Section 5 supplies counting and explicit weighted-modular constructions.

3. **Count shared computation.** The potential weights parity and nonlinear gates differently and gives credits at two successive nonlinear gates on suitable single-successor chains. A critical elimination step creates such a chain; its credits pay for a case that otherwise limits the bound. Combining this with Schnorr's algebraic-degree bound [15] gives 17/7.

The proof builds on established weighted measures and fan-out credits [9–14]. Its contribution is the particular two-level accounting for the full binary basis, the dense SAT embedding, and the resulting theorem. It applies to one shared circuit for all the tests, without assuming independent clause evaluation.

**Scope.** There are exactly $m = 2^w - 1$ clauses over $m$ available variables. The bound is asymptotic along the lengths $n = 3m(w + 1)$, includes a lower-order loss, and does not assert $17n/7$ at every finite size. Theorem 8.2 gives the finite parameter inequality.

## 2 The input and the circuit

**Definition 2.1** (Encoding parameters)**.** Fix an integer $w \geq 5$. There are $m = 2^w - 1$ available variables $X_1, \ldots, X_m$ and exactly $m$ clauses.

**Definition 2.2** (Literal field)**.** For the variables of Definition 2.1, a field consists of identifier bits $b_0, \ldots, b_{w-1}$ followed by a sign bit $s$. The identifier is $v = \sum_{j=0}^{w-1} 2^j b_j$. Identifier zero is invalid. Each other identifier denotes $X_v$ when $s = 0$ and $\neg X_v$ when $s = 1$.

**Definition 2.3** (Clause-by-clause encoding)**.** In each of the $m$ ordered records of Definition 2.1, concatenate three fields of Definition 2.2 in literal order. Concatenating the records gives $n = 3m(w+1)$ bits. Variables may be absent. Repeated literals, repeated clauses and tautologies are allowed.

**Definition 2.4** (Decision function)**.** On the $n$ bits of Definition 2.3, $F_w$ is zero if any identifier is invalid. Otherwise it is one exactly when some assignment to the $m$ variables satisfies all the clauses.

**Definition 2.5** (Binary circuits)**.** A $B_2$ circuit is a finite acyclic network with labelled primary inputs and a designated output. Each gate computes an arbitrary Boolean function of at most two predecessor values. A predecessor may supply both arguments. Depth, sharing and the choice of circuit at each length are unrestricted.

**Definition 2.6** (Size)**.** For a circuit $C$ of Definition 2.5, $|C|$ counts every internal gate, including unary and constant gates. Inputs and wires contribute zero.

**Definition 2.7** (Circuit complexity)**.** The circuit complexity $C_{B_2}(f)$ is the minimum size in Definition 2.6 among circuits of Definition 2.5 whose output equals $f$ on every input.

**Definition 2.8** (Essential input)**.** An input is essential if two assignments differing only there can have different outputs.

**Definition 2.9** (Constant restriction)**.** A constant restriction $\rho$ fixes a specified set of input bits to specified values.

**Definition 2.10** (Cofactor)**.** For the restriction $\rho$ of Definition 2.9, $f|_\rho$ denotes the resulting function on all remaining bits. It is called a cofactor of $f$. Fixing a specified pair gives four cofactors, some of which may coincide.

**Lemma 2.11** (Elementary size bound)**.** *A circuit of Definition 2.5 with $N \geq 1$ essential inputs in the sense of Definition 2.8 has at least $N - 1$ gates.*

*Proof.* Keep only the vertices that reach the output. Every essential input remains. If $s$ gates and $a \geq N$ inputs remain, the underlying undirected graph is connected, so it contains at least $a + s - 1$ edges. Each gate receives at most two wires, giving $a + s - 1 \leq 2s$. □

# 3 Embedding a Boolean test in the formula

To make a literal field test membership in a prescribed set of variable numbers, we use the other clauses of Definition 2.3 to enforce a common assignment on that set. For $a \in \{0,1\}^r$ we also write $a$ for the unsigned integer represented by its bits in the order of Definition 2.2.

**Lemma 3.1** (Sparse-helper restriction for an arbitrary block)**.** *For $F_w$ of Definition 2.4 and restrictions as in Definition 2.9, let $1 \leq r \leq w - 1$ and let $g : \{0,1\}^r \to \{0,1\}$ be any function. Put $M = m - |g^{-1}(0)|$ and assume $M \geq 1$. There is a restriction fixing exactly $n - 3Mr$ input bits of $F_w$ whose restricted function is*

$$G = \bigwedge_{c=1}^{M} \left(g(x_{c,1}) \vee g(x_{c,2}) \vee g(x_{c,3})\right)$$

*on $L = 3Mr$ free bits, the $x_{c,j} \in \{0,1\}^r$ being disjoint blocks of input bits.*

*Proof.* Call $v = 2^{w-1} + a$ forbidden if $g(a) = 0$. Use one clause with three copies of the negative literal $(\neg X_v \vee \neg X_v \vee \neg X_v)$ for each forbidden identifier $v$ (this uses $|g^{-1}(0)|$ clause records). The remaining $M$ records are positive clauses whose three identifier fields have the highest bit set to one, positions $r$ through $w-2$ set to zero, and their lowest $r$ bits free. Set each of their sign bits to zero. The fixed helper clauses and these choices fix exactly $n - 3Mr$ bits. The negative clauses force every variable with a forbidden identifier false. If every positive clause contains a non-forbidden identifier, setting all non-forbidden variables true satisfies the formula; if some positive clause has three forbidden identifiers it is unsatisfiable. All identifiers are valid (they lie between $2^{w-1} \geq 1$ and $2^{w-1} + 2^r - 1 \leq m$). Hence the restricted function is $G$. □

**Lemma 3.2** (Essentiality of every encoding bit)**.** *Every bit in Definition 2.3 is essential to $F_w$ of Definition 2.4 in the sense of Definition 2.8.*

*Proof.* Choose any clause record and any of its three fields. For both witnesses, fill unused records with tautologies $(X_1 \vee \neg X_1 \vee X_1)$.

For a selected sign bit, let its literal initially be $X_2$ and the other two literals in its clause both be $X_1$. Elsewhere add the two repeated-unit clauses $(\neg X_1 \vee \neg X_1 \vee \neg X_1)$ and $(\neg X_2 \vee \neg X_2 \vee \neg X_2)$. With the selected sign positive, the formula is unsatisfiable, since $X_1 = X_2 = 0$ are forced. Flipping that one sign makes the selected literal $\neg X_2$, hence the formula satisfiable.

For a selected identifier bit at binary position $j \in \{0, \ldots, w-1\}$, let the selected literal and its two companions all initially be positive $X_m$, and elsewhere add the unit clause $(\neg X_m \vee \neg X_m \vee \neg X_m)$. This makes the formula unsatisfiable. Flip only the designated identifier bit: since $m = 2^w - 1$ has all binary digits 1, its identifier becomes $m - 2^j \geq 1$, distinct from $m$. Now set $X_m = 0$ and $X_{m-2^j} = 1$, satisfying the selected clause and all other non-tautological clauses. Both identifiers are valid.

The construction works in any selected record and field because $m \geq 3$ leaves enough other records for the unit clauses. Hence each input bit has a witness to its essentiality. □

Lemmas 2.11 and 3.2 give the baseline $n - 1$. To improve it we must count both the circuit left by Lemma 3.1 and the gates that fixing encoding bits removes.

# 4 Transferring a lower bound through a restriction

**Definition 4.1** (Gate types)**.** Classify a gate of Definition 2.5 by its truth table on its formal arguments. An essential binary table depends on both arguments as in Definition 2.8. Two such tables are parity, $x \oplus y \oplus c$, where $\oplus$ is exclusive OR. The other eight are nonlinear, $((x \oplus a) \wedge (y \oplus b)) \oplus c$ for bits $a, b, c$. Indeed the nonlinear tables have either one or three ones, and the remaining essential tables are parity. A degenerate table is constant or depends on only one argument. Write $X(C)$ and $A(C)$ for the numbers of parity and nonlinear gates.

**Definition 4.2** (Blocking value)**.** For a nonlinear gate in Definition 4.1, the blocking value at an operand makes the output constant for both values of the other operand. In the displayed nonlinear form these values are $a$ and $b$, respectively, and the constant output is $c$.

**Definition 4.3** (Normalized circuit)**.** A circuit of Definition 2.5 is normalized if all gates have essential binary tables in Definition 4.1, their two predecessors are distinct, every gate reaches the output, and there is no input triangle. An input triangle consists of gates $P = p(x, u)$ and $Q = q(x, P)$ for a primary input $x$ and another predecessor $u$.

Triangle removal and the simplification of constant and unary gates are standard operations in gate elimination [12,14].

**Lemma 4.4** (Normalization)**.** *If an output has at least two essential inputs as in Definition 2.8, it has a normalized circuit in the sense of Definition 4.3 of no greater size than the original.*

*Proof.* Prune gates that do not reach the output. A constant operand, a repeated operand or a degenerate table leaves a constant or a literal of a predecessor. Substitute the constant or bypass the gate, absorbing any negation into the receiving tables. The surviving output is a literal of a binary gate since it has at least two essential inputs. Designate that gate as output, prune its successors and absorb any final negation into its own table.

For an input triangle as in Definition 4.3, replace the instruction at $Q$ by $\psi(x, u) = q(x, p(x, u))$. Retain $P$ if another gate still uses it. If $q$ is nonlinear, blocking it at $x$ gives a constant cofactor. If $q$ is parity and $p$ is nonlinear, blocking $p$ at $x$ also gives a constant cofactor. If both are parity, the two occurrences of $x$ cancel. In each case $\psi$ is nonlinear or degenerate. Simplify and prune again. In an ordering in which predecessors come first, the triangle replacement substitutes the earlier predecessor $u$ for $P$. Between deletions the sum of predecessor indices over all wires therefore strictly decreases. The procedure terminates, and no operation adds gates. □

**Lemma 4.5** (Gates removed by a restriction)**.** *In a normalized circuit of Definition 4.3, fix $t$ inputs that were essential before restriction. Suppose the cofactor of Definition 2.9 has at least two essential inputs. Simplification removes at least $t$ gates.*

*Proof.* Let $d$ be the total number of wires leaving the fixed inputs. Every essential input has a path to the output, so $d \geq t$. Form a set consisting of the fixed inputs and all $h$ gates made constant by propagating constants. The output lies outside this set. Let $e$ count wires within it and $b$ count wires leaving it. Each of its gates has two incoming wires and at least one outgoing wire. Hence $e \leq 2h$ and $e + b \geq d + h$, giving $h + b \geq d$. The $b$ leaving wires enter distinct gates since two constant operands would have put the receiving gate in the set. All $h$ constant gates and all $b$ receiving gates can be removed by the substitutions of Lemma 4.4, including when a receiving gate is the output. □

**Lemma 4.6** (Transfer)**.** *With $C_{B_2}$ as in Definition 2.7, if a restriction fixing $n - L$ bits of $F_w$ yields a function $G$ depending on all of its $L \geq 2$ free bits, then $C_{B_2}(F_w) \geq n - L + C_{B_2}(G)$.*

*Proof.* Normalize a minimum circuit by Lemma 4.4. Every fixed bit is essential for $F_w$ (Lemma 3.2), so it has an out-edge in a normalized circuit for $F_w$, and Lemma 4.5 deletes at least $n - L$ gates; the rest computes $G$. □

# 5 Boolean tests that survive restrictions

Lemma 4.6 reduces the problem to lower-bounding the size of a circuit for the free fields. We require two properties of each field's test. High algebraic degree forces nonlinear gates. Distinct cofactors make it possible to eliminate gates repeatedly.

**Definition 5.1** (Algebraic degree)**.** Represent a Boolean function by its unique multilinear polynomial over the two-element field $\mathbb{F}_2$. Its algebraic degree $\deg_{\mathbb{F}_2} f$ is the largest number of variables in a monomial with nonzero coefficient, with degree zero for a constant. On $r$ variables, full degree means degree $r$. We use the standard multilinear representation [17, Section 2.2.1].

**Lemma 5.2** (Test for full degree)**.** *For $r \geq 1$, an $r$-variable function has full degree in Definition 5.1 exactly when it has an odd number of accepting assignments.*

*Proof.* Sum the multilinear polynomial over the Boolean cube in $\mathbb{F}_2$. A monomial involving fewer than $r$ variables is counted an even number of times. The unique possible monomial of degree $r$ contributes exactly its coefficient. The sum also equals the parity of the number of accepting assignments. □

**Definition 5.3** (Read-once composition)**.** A block is a Boolean function on its own set of inputs. A read-once AND/OR composition is a rooted binary tree of AND and OR operations whose leaves are blocks on pairwise disjoint sets of inputs. Each block occurs once. Write $B$ for the number of leaves and $L$ for the total number of inputs. The tree specifies a function. Circuits computing that function may share gates as in Definition 2.5.

Schnorr's bound on the number of multiplications applies to nonlinear binary gates [15,11].

**Lemma 5.4** (Degree requires nonlinear gates)**.** *If $f$ has algebraic degree $D \geq 1$ as in Definition 5.1, every circuit of Definition 2.5 computing $f$ contains at least $D-1$ nonlinear gates in the sense of Definition 4.1.*

*Proof.* Topologically enumerate the nonlinear gates $z_1, \ldots, z_A$. Regard their outputs temporarily as independent formal Boolean variables. All other gates compute affine functions over $\mathbb{F}_2$, so every signal between nonlinear gates is affine in the original inputs and the previously introduced $z_j$'s. Thus the polynomial $p_j$ defining a nonlinear $z_j$ has total degree at most two in original inputs and earlier $z$ symbols, and the final output is affine in those variables.

Work in the Boolean polynomial quotient, imposing $u^2 = u$ for every current variable $u$, including formal gate variables. Equivalently, multilinearize after each substitution, rather than only after the final substitution. Eliminate $z_A, z_{A-1}, \ldots, z_1$ in reverse order. If the current multilinear polynomial $P$ has degree at most $d$, express it as

$$P = z_j U + V, \qquad z_j \notin \operatorname{Var}(U) \cup \operatorname{Var}(V), \quad \deg U \leq d-1, \quad \deg V \leq d.$$

Substituting $p_j$ gives $p_j U + V$, of degree at most $d+1$; multilinearization cannot raise degree. The initial output polynomial has degree at most one, so after the $A$ successive eliminations the output polynomial in the original inputs has degree at most $A+1$. Therefore $D \leq A+1$ and $A \geq D-1$. □

**Proposition 5.5** (Degree of a read-once composition)**.** *A composition as in Definition 5.3 of functions of degrees $d_1, \ldots, d_B \geq 1$ on disjoint inputs has degree $d_1 + \cdots + d_B$. In particular a composition of full-degree blocks of total length $L$ has degree $L$, and every circuit for it has at least $L-1$ nonlinear gates.*

*Proof.* For disjoint-variable functions $f, g$ of degrees $d, e \geq 1$, the product $fg$ has degree $d+e$, because its top-degree terms are products of monomials on disjoint variable sets and cannot cancel; and $f \vee g = f + g + fg$ has the same unique top-degree part. Induct over the tree and apply Lemma 5.4. □

**Definition 5.6** (Cofactor conditions)**.** Use essential inputs and cofactors as in Definitions 2.8 and 2.10. Write (E1) if every restriction fixing one bit is essential in every remaining bit. Write (E2) if this holds after fixing any set of at most two bits. Write (S4) if the four cofactors at every pair of distinct inputs are pairwise distinct. We use (E2) before a restriction and (E1) on the resulting function.

**Definition 5.7** (Robust block)**.** For integers $3 \leq k \leq r$, an $r$-input block is $k$-robust if all its restrictions leaving at least $k$ inputs satisfy (S4) of Definition 5.6. This is strong two-dependence in the sense of Lachish and Raz [9, Definitions 3.1–3.2]: equality of two cofactors persists under every common further restriction, so testing restrictions down to exactly $k$ inputs suffices.

**Lemma 5.8** (Cofactors across blocks)**.** *In a composition of Definition 5.3, suppose every leaf is a restriction of a $k$-robust block as in Definition 5.7. If each leaf retains at least $k+2$ bits, the composition has (E1), (E2) and (S4) of Definition 5.6. If each retains at least $k+1$ bits, the composition still has (E1).*

*Proof.* On at least $k$ remaining bits every leaf is essential in every bit. Otherwise fixing an inessential bit either way would give identical cofactors for a pair containing it. Each leaf therefore takes both values. Induction from the leaves shows that each subtree can take either value. For either parent operation, assign both children the desired value. Their input sets are disjoint. To expose a selected leaf, set the sibling at each ancestor to one for an AND parent and zero for an OR parent. The output now equals that leaf. This remains possible after any restriction leaving at least $k$ bits in each leaf, proving the assertions about essentiality.

To verify (S4), first take two bits in one leaf. Exposing the leaf preserves its four distinct cofactors. For bits in separate leaves, set all subtrees branching off the two paths to neutral values as above. The resulting output combines the selected leaves by their first common ancestor's operation. Each selected bit has two distinct nonconstant cofactors. Distinctness follows by essentiality before fixing the bit, and nonconstancy by essentiality afterward. For AND, the accepting inputs of each pair cofactor form the Cartesian product of the two accepting sets. The products are nonempty, and their coordinate projections recover both factors, so all four are distinct. For OR apply the same projection identity to the rejecting sets. This is the coordinate-rectangle argument of communication complexity [18, Section 1.2]. □

### 5.1 Constructing full-degree robust blocks

We give two choices for the Boolean test in Lemma 3.1. The first keeps the threshold $k$ logarithmic in the block length. The second specifies every block by an explicit modular sum. The counting argument for strong two-dependence is due to Lachish and Raz [9, Section 3.2]. We impose full degree simultaneously.

**Proposition 5.9** (Counting construction)**.** *For $r \geq k+2$ and $k \geq 3$ as in Definition 5.7, if $2^{k-2} > \log_2(6r(r-1)) + r\log_2 3$, a $k$-robust function on $r$ inputs of full $\mathbb{F}_2$-degree $r$ exists.*

*Proof.* For a uniformly random $g$, the four cofactors of a pair in a restriction with $j$ free inputs are the restrictions of $g$ to four disjoint subcubes, hence independent uniform functions on $j-2$ inputs, so they fail to be distinct with probability at most $6 \cdot 2^{-2^{j-2}} \leq 6 \cdot 2^{-2^{k-2}}$ for $j \geq k$. There are at most $3^r$ restrictions and $\binom{r}{2}$ pairs, so $g$ fails to be $k$-robust with probability at most $3^r \cdot 3r(r-1) \cdot 2^{-2^{k-2}} < \frac{1}{2}$. By Lemma 5.2, the degree is $r$ exactly when the number of ones of $g$ is odd, which has probability $\frac{1}{2}$. Hence a random $g$ is $k$-robust and of degree $r$ with positive probability. □

**Proposition 5.10** (Weighted modular blocks)**.** *Let $r \geq 4$, let $p$ be the least prime greater than $2r$, let $m_p$ be the least integer with $1 + m_p(m_p+1)/2 \geq p$, and let $k = m_p + 2 \leq \lceil\sqrt{8r}\rceil + 2$. For $r \geq k+2$, in the terminology of Definitions 5.1 and 5.7, for every residue $t$ the function*

$$g_{r,t}(a_1, \ldots, a_r) = \left[\sum_{i=1}^{r} i a_i \equiv t \pmod{p}\right]$$

*is $k$-robust, and for some residue $t$ it has full degree $r$.*

*Proof. Robustness.* Consider a restriction with free set $F$, $|F| \geq k$, and residual target $\tau$; it is $[\sum_{l \in F} l a_l \equiv \tau]$. For a pair $i, j \in F$ the four cofactors are $[S \equiv \tau - \sigma]$ for $\sigma \in \{0, i, j, i+j\}$, where $S$ ranges over the subset sums of $F \setminus \{i, j\}$. The four targets are distinct modulo $p$ because $1 \leq i \neq j \leq r$ and $i + j < p$. The set $H = F \setminus \{i, j\}$ consists of at least $m_p$ distinct residues in $[1, r]$, and $H \cap (-H) = \varnothing$ because $2r < p$. By Balandraud's theorem [19] (for an odd prime $p$ and $H \subseteq \mathbb{Z}/p\mathbb{Z}$ with $H \cap (-H) = \varnothing$, the set of subset sums of $H$, the empty sum included, has at least $\min(p, 1 + |H|(|H|+1)/2)$ elements) the subset sums of $H$ cover $\mathbb{Z}/p\mathbb{Z}$, so every target is attained: the four cofactors are nonzero with pairwise disjoint supports, hence distinct. The bound on $k$ follows from $p < 4r$, which holds by Bertrand's postulate [20] (there is a prime between $2r$ and $4r$): with $q = \lceil\sqrt{8r}\rceil$, $q(q+1)/2 > 4r > p - 1$, so $m_p \leq q$.

*Degree.* By Lemma 5.2, the degree is $r$ exactly when $N_t = \#\{a : \sum ia_i \equiv t\}$ is odd. Over GF(2), $\sum_t N_t z^t \equiv \prod_{i=1}^{r}(1+z^i) \pmod{z^p-1}$. The polynomial $z^p-1 = (z+1)\Phi_p(z)$ is squarefree over GF(2), and every irreducible factor of $\Phi_p$ has primitive $p$-th roots of unity as roots, so it divides $1+z^i$ only if $p \mid i$, which fails for $1 \le i \le r < p$. Hence $\Phi_p$ does not divide the product, the product is nonzero modulo $z^p-1$, and some $N_t$ is odd.

*Finding $t$.* The parities of all $N_t$ are the coefficients of $\prod_{i=1}^{r}(1+z^i)$ in $\mathrm{GF}(2)[z]/(z^p-1)$, computed by $r$ steps $v \leftarrow v + z^i v$ in $O(rp) = O(r^2)$ bit operations. Choose the least residue with odd count. The recurrence specifies the residue and the block in polynomial time in $r$. □

# 6 Counting gates under a restriction

A bit used by just one gate can become cheaper to eliminate later. We keep track of that possibility explicitly, including the case where the gate itself has just one user. First we specify how gate identities and wires are tracked through simplification.

**Definition 6.1** (Successor set and fan-out)**.** For a vertex $v$ in Definition 2.5, $\mathrm{succ}(v)$ is the set of gates using $v$ as an operand and $d(v) = |\mathrm{succ}(v)|$ is its fan-out. For a set $Y$ of vertices let $\mathrm{succ}(Y) = \bigcup_{v \in Y} \mathrm{succ}(v)$. Counts refer to the current circuit, including any gates temporarily present that have no path to the output.

**Definition 6.2** (Resolution under a restriction)**.** Let $C$ be a circuit computing $f$ whose gates have two distinct predecessors and whose output gate has no successors. Intermediate gates may have degenerate tables in Definition 4.1. Use a restriction $\rho$ of Definition 2.9 that fixes the set $I$ and leaves two or more essential inputs. Process the nodes in topological order and assign to every node $v$ a resolved signal $\sigma(v)$, which is a constant or a literal (a node or its complement): a fixed input resolves to its constant and a free input $z$ to $z$. For a gate $G = g(a, b)$ consider the function $g(\sigma(a), \sigma(b))$, which depends on at most the nodes underlying $\sigma(a)$ and $\sigma(b)$.

- If it is constant, $G$ becomes constant, and $\sigma(G)$ is that constant.
- If it depends on exactly one node $N$, then $\sigma(G)$ is the corresponding literal of $N$; we say that $G$ is bypassed and passes $N$.
- Otherwise it depends on two distinct nodes; then $G$ survives resolution, $\sigma(G) = G$, its operands are replaced by these two nodes, and the complements are absorbed into its table.

This one rule covers a constant operand, two operands resolving to literals of the same node (a collapse, which may produce a constant), and a degenerate table.

**Definition 6.3** (Simplified circuit)**.** Using the resolved signals of Definition 6.2, suppose $f|_\rho$ depends on at least two inputs. The resolved signal of the output gate of $C$ is a literal of a gate $N$ that survives resolution. Designate $N$ as the output (its table is complemented if the literal is negated), and finally all gates without a path to $N$ are pruned. In particular all successors of $N$ are pruned, so the complementation affects no remaining gate. The result is the simplified circuit $C^\rho$. A gate of $C$ is eliminated if it is not a gate of $C^\rho$, that is, if it becomes constant, is bypassed, or survives resolution and is pruned. Write $N(C^\rho)$ for the result of normalizing $C^\rho$ by Lemma 4.4.

**Lemma 6.4** (Resolution and dependence)**.** *The simplified circuit in Definition 6.3 computes the cofactor and preserves the type of every surviving gate. If every path from a free input to the output meets a gate constant on the restriction, the cofactor is independent of that input.*

*Proof.* Induct in predecessor order. The table at each gate is evaluated on exactly its resolved operands. When two distinct operands survive, the only changes to the table are input or output complements, which preserve both essential gate types in Definition 4.1. Replacing every constant gate on the stated paths by its value disconnects the free input from the output. □

**Lemma 6.5** (Passing)**.** *Using the successor sets and simplified circuit of Definitions 6.1 and 6.3, let $N$ be a free input or a gate that survives resolution, and let $S_N$ be the set of nodes whose resolved signal is a literal of $N$ (so $N \in S_N$).*

*(a) Every node of $S_N \setminus \{N\}$ is a gate having an operand in $S_N$.*

*(b) The successors of $N$ in $C^\rho$ are exactly the gates of $C^\rho$ having an operand (in $C$) in $S_N$.*

*(c) If $N \in Y \subseteq S_N$ and no gate of $\mathrm{succ}(Y) \setminus Y$ lies in $S_N$, then $S_N = Y$.*

*Proof.* (a) A node other than $N$ whose resolved signal is a literal of $N$ is not an input, so it is a gate $g(a, b)$ whose function $g(\sigma(a), \sigma(b))$ depends on $N$; hence $\sigma(a)$ or $\sigma(b)$ is a literal of $N$. (b) The operands of a gate of $C^\rho$ are the nodes underlying the resolved signals of its operands in $C$. (c) Otherwise let $G$ be the first node of $S_N \setminus Y$ in topological order. By (a), $G$ has an operand in $S_N$; this operand precedes $G$, so it lies in $Y$, and $G \in \mathrm{succ}(Y) \setminus Y$, a contradiction. □

**Definition 6.6** (First credit)**.** A primary input $z$ earns one credit if its successor set in Definition 6.1 consists of one nonlinear gate $W_z$. Call $z$ a $D$-variable and $W_z$ its $D$-gate. Denote the set of such inputs by $\mathcal{D}(C)$ and its cardinality by $D(C)$.

**Lemma 6.7** (Forbidden configurations)**.** *Use the credits of Definition 6.6 and the hypotheses of Definition 5.6 for the function $f$ computed by $C$.*

*(a) If $f$ has (E1), no gate both of whose operands are primary inputs is a $D$-gate. In particular distinct $D$-variables have distinct $D$-gates, and $D(C) \le A(C)$.*

*(b) If $f$ has (E2) and $T$ is a gate with an essential table whose operands are primary inputs $x, y$, then no successor of $T$ is a $D$-gate.*

*Proof.* (a) Let $W = W_z$ have the operands $z$ and a primary input $v \neq z$. Fix $v$ to its blocking value for $W$. Then $W$ is constant, and every path from $z$ to the output starts with the edge $z \to W$, because $W$ is the only successor of $z$; by the disconnection principle $f|_v$ does not depend on $z$, contradicting (E1). Two $D$-variables with a common $D$-gate would make it a gate with two primary operands; and $D$-gates are nonlinear, so $D(C) \le A(C)$. (b) Suppose a successor $W$ of $T$ is the $D$-gate of $z$. Then the operands of $W$ are $T$ and $z$, and $z \notin \{x, y\}$ since the only successor of $z$ is $W \neq T$. As $T$ is essential, it takes both values, so some assignment $(x, y) = (a, b)$ gives $T$ the value blocking $W$ at its $T$-operand. Under that assignment $W$ is constant and $z$ is disconnected, contradicting (E2). □

**Definition 6.8** (Second credit)**.** An input $z \in \mathcal{D}(C)$ from Definition 6.6 earns a second credit when $\mathrm{succ}(W_z) = \{W_z^2\}$ and $W_z^2$ is nonlinear. Call such an input a $D^*$-variable. Denote these inputs by $\mathcal{D}^*(C)$ and their number by $D^*(C)$. A gate plays a first or second role for $z$ if it is $W_z$ or $W_z^2$, respectively.

**Definition 6.9** (Two-level potential)**.** With gate counts from Definition 4.1 and credits from Definitions 6.6 and 6.8, put

$$\Psi_2(C) = 7X(C) + 4A(C) - D(C) - D^*(C). \tag{1}$$

**Lemma 6.10** (Roles)**.** *For roles as in Definition 6.8 and conditions as in Definition 5.6, let $C$ compute $f$.*

*(a) Under (E1), no gate having a primary input as an operand plays the role $W^2$, and a direct successor of a primary input $x$ with $d(x) \geq 2$ plays no role.*

*(b) Under (E2), a successor of a gate $T$ with an essential table and two primary operands plays no role.*

*Proof.* (a) Let $G$ have the primary operand $x$, and suppose $G = W_z^2$. Then $W_z$ is an operand of $G$ and $G$ is its only successor; $W_z \neq x$ because $W_z$ is a gate, so $G = g(x, W_z)$. Also $z \neq x$, since the only successor of $z$ is $W_z \neq G$. Fixing $x$ to its blocking value for $G$ makes $G$ constant, and every path from $z$ to the output passes through $W_z$ and then $G$; so $f|_x$ does not depend on $z$, contradicting (E1). If moreover $d(x) \geq 2$, then $G$ is not $W_z$ for any $z$: $z$ would be the other operand of $G$, a primary input, and $G$ would have two primary operands, contrary to Lemma 6.7(a). (b) Denote the two primary operands of $T$ by $x, y$. The role $W$ is excluded by Lemma 6.7(b). If a successor $S$ of $T$ were $W_z^2$, its operands would be $T$ and $W_z$ ($W_z \neq T$ by Lemma 6.7(a)), and $z \notin \{x, y\}$ because the only successor of $z$ is $W_z \neq T$; an assignment of $(x, y)$ giving $T$ the value blocking $S$ makes $S$ constant and disconnects $z$, contradicting (E2). □

**Lemma 6.11** (Charging for $\Psi_2$)**.** *Use the potential of Definition 6.9 and resolution of Definition 6.2. Let $C$ compute $f$ with (E1) of Definition 5.6, and let $\rho$ fix a set $I$ of inputs such that $f|_\rho$ depends on every free input (and on at least two inputs). For an input $z$ use $[P] = 1$ if $P$ holds and $[P] = 0$ otherwise. Let $\lambda(z) = [z \in \mathcal{D}(C)] + [z \in \mathcal{D}^*(C)]$, and let $\lambda'(z) = [z \in \mathcal{D}(C^\rho)] + [z \in \mathcal{D}^*(C^\rho)]$ if $z \notin I$ and $\lambda'(z) = 0$ if $z \in I$. Put $\Gamma = \sum_z \max(\lambda'(z) - \lambda(z), 0)$ (the gains). With $w_2(G) = 7$ for parity, 4 for nonlinear and 0 for degenerate gates, there are integers $\chi_2(G) \geq 0$ on the eliminated gates (the charges) such that*

$$\Psi_2(C) - \Psi_2(C^\rho) = \sum_{G \text{ eliminated}} \big(w_2(G) - \chi_2(G)\big) + \Gamma,$$

*only nonlinear gates that play a role are charged, and $\chi_2(G) \leq 2$ for every gate. If $I$ contains no $D$-variable of $C$, then $\chi_2(G) \leq 1$ for every gate and no charged gate becomes constant.*

*Proof.* By Lemma 6.4, $7X + 4A$ drops by exactly $\sum_{G \text{ eliminated}} w_2(G)$, and $D + D^*$ drops by $\sum_z(\lambda(z) - \lambda'(z)) = \Lambda - \Gamma$, where $\Lambda = \sum_z \max(\lambda(z) - \lambda'(z), 0)$ is the total loss. We distribute the $\Lambda$ units of loss over eliminated gates. Let $\lambda(z) > \lambda'(z)$; then $z \in \mathcal{D}(C)$.

*Case $z \notin I$ and $W_z$ a gate of $C^\rho$.* Since $W_z$ survives, its resolved signal is itself. Lemma 6.5(c) with $Y = \{z\}$ gives $S_z = \{z\}$, and Lemma 6.5(b) makes $W_z$ the only successor of $z$ in $C^\rho$. Its type is preserved by Lemma 6.4. Thus $z \in \mathcal{D}^*(C) \setminus \mathcal{D}^*(C^\rho)$ and the loss is 1. Then $W_z^2$ is eliminated: otherwise $W_z^2$, the only gate of $\mathrm{succ}(W_z)$, survives resolution, so $S_{W_z} = \{W_z\}$ by Lemma 6.5(c), the only successor of $W_z$ in $C^\rho$ is $W_z^2$ (Lemma 6.5(b)), still nonlinear, and $z$ would remain a $D^*$-variable. Charge $W_z^2$ once.

*Case $z \notin I$ and $W_z$ eliminated.* If $z$ is a $D$-variable of $C^\rho$, the loss is 1 ($z$ only leaves $D^*$); charge $W_z$ once. Otherwise the loss is $\lambda(z)$; charge $W_z$ once and, if $z \in \mathcal{D}^*(C)$, also $W_z^2$ once. In the latter case $W_z^2$ is eliminated. Indeed, suppose not. First, $W_z \in S_z$: otherwise Lemma 6.5(c) with $Y = \{z\}$ gives $S_z = \{z\}$, and $z$ would have no successor in $C^\rho$ (its only successor in $C$ is eliminated), although $f|_\rho$ depends on $z$. Then $Y = \{z, W_z\} \subseteq S_z$, and the only gate of $\mathrm{succ}(Y) \setminus Y$ is $W_z^2$, which survives resolution; so $S_z = \{z, W_z\}$, and by Lemma 6.5(b) the only successor of $z$ in $C^\rho$ is the nonlinear gate $W_z^2$, i.e. $z$ is a $D$-variable of $C^\rho$, a contradiction.

*Case $z \in I$.* The loss is $\lambda(z)$. Charge $W_z$ (eliminated, as it has a constant operand) once and, if $z \in \mathcal{D}^*(C)$, charge $W_z^2$ once if it is eliminated and $W_z$ a second time otherwise.

In all cases exactly $\lambda(z) - \lambda'(z)$ units are charged, each to an eliminated gate that plays a role for $z$ in $C$. Hence $\Lambda = \sum_G \chi_2(G)$, which gives the identity, and charged gates are nonlinear.

*Bounds.* Let $G$ be a gate. If $G$ plays the role $W$ for some $z$, it has the primary operand $z$, so it plays no role $W^2$ (Lemma 6.10(a)), and $z$ is unique (Lemma 6.7(a)). Then $G$ is charged only by $z$: at most once if $z \notin I$, and at most twice if $z \in I$. Otherwise $G$ plays only roles $W^2$, for at

most two variables (for each of them $W_z$ is an operand of $G$, and $W_z$ determines $z$ by Lemma 6.7(a)), and each variable charges $G$ at most once. Hence $\chi_2(G) \le 2$.

Now let $I$ contain no $D$-variable. Then only a gate $G = g(W_{z_1}, W_{z_2})$ that is $W_{z_1}^2$ and $W_{z_2}^2$ for $z_1 \neq z_2$ could be charged twice; we show that such a gate is not eliminated. For $i = 1, 2$, $z_i$ is free. $W_{z_i}$ does not become constant, since every path from $z_i$ to the output passes through $W_{z_i}$ and $f|_\rho$ depends on $z_i$; and it does not pass a node other than $z_i$, since otherwise $W_{z_i} \notin S_{z_i}$, so $S_{z_i} = \{z_i\}$ by Lemma 6.5(c), and $z_i$ would have no successor in $C^\rho$. So $\sigma(W_{z_i})$ is $W_{z_i}$ or a literal of $z_i$. The four nodes $z_1, z_2, W_{z_1}, W_{z_2}$ are distinct (Lemma 6.7(a)), so the operands of $G$ resolve to literals of two distinct nodes, and $G$ survives resolution. It is not pruned. Indeed, $S_{z_1} \subseteq \{z_1, W_{z_1}\}$ by Lemma 6.5(c) (applied with $Y = \{z_1\}$ if $W_{z_1} \notin S_{z_1}$ and with $Y = \{z_1, W_{z_1}\}$ otherwise; the only gate of $\mathrm{succ}(\{z_1, W_{z_1}\})$ outside $\{z_1, W_{z_1}\}$ is $G$, which survives), and if $W_{z_1}$ survives resolution then $S_{W_{z_1}} = \{W_{z_1}\}$ in the same way. By Lemma 6.5(b), in $C^\rho$ the successors of $z_1$ lie in $\{W_{z_1}, G\}$ and those of $W_{z_1}$ in $\{G\}$. Moreover $W_{z_1}$ is not the output of $C^\rho$: otherwise the output gate of $C$ would lie in $S_{W_{z_1}} = \{W_{z_1}\}$, but $W_{z_1}$ has the successor $G$ in $C$, while the output gate of $C$ has none. As $f|_\rho$ depends on $z_1$, some path leads from $z_1$ to the output of $C^\rho$, and it passes through $G$; so $G$ is a gate of $C^\rho$. Hence $\chi_2(G) \le 1$ for every gate. Finally, every charged gate then plays a role for a free variable $z$, and a gate $W_z$ or $W_z^2$ that becomes constant would disconnect $z$; so no charged gate becomes constant. □

**Lemma 6.12** (Normalization preserves the potential bound)**.** *Suppose all gates of $C'$ are essential binary gates with distinct predecessors and reach the output. If its function has at least two inputs and satisfies (E1) of Definition 5.6, then $\Psi_2(N(C')) \le \Psi_2(C')$ for the potential of Definition 6.9.*

*Proof.* Consider one triangle replacement from Lemma 4.4. For $P = p(x, u)$ and $Q = q(x, P)$, replace $Q$ by $\psi(x, u) = q(x, p(x, u))$ and retain $P$ temporarily. Only the operand edge $P \to Q$ is replaced by $u \to Q$. As in Lemma 4.4, $\psi$ is nonlinear or degenerate, so the gate weight $7X + 4A$ cannot increase.

A $D$-variable $z$ cannot be $u$, since its unique successor would then be $P$, with two primary operands, contrary to Lemma 6.7(a). Nor can $Q$ be its $D$-gate: $Q$ has just one primary operand $x$, and $x$ feeds both $P$ and $Q$. Thus no first credit is lost. For a second credit of $z$, the added edge would change $\mathrm{succ}(W_z)$ only if $u = W_z$, making $P = W_z^2$ a gate with a primary operand, excluded by Lemma 6.10(a). The removed edge would change that successor set only if $P = W_z$, but its operand $x$ has two successors and its other operand $u$ cannot be a $D$-variable. Finally $Q$ cannot be $W_z^2$ by Lemma 6.10(a). Both credits persist.

Resolve the modified circuit as in Definition 6.2 with no fixed inputs. Its function is unchanged and depends on every input by (E1). Lemma 6.11 assigns at most one unit of charge to an eliminated nonlinear gate, and no charge to a degenerate gate. Resolution cannot increase the potential. Repeat the replacement and resolution until termination as in Lemma 4.4. □

# 7 Fourteen units per input

The four-cofactor condition from Definition 5.6 rules out one local pattern that can occur when only three cofactors are distinct. The second credit from Definition 6.8 supplies the last two units in the remaining tight case. The argument refines the fan-out analysis of Zwick [12, Lemma 3.2] for the gate types of Definition 4.1.

**Lemma 7.1** (One step of elimination)**.** *Let $C$ be normalized in the sense of Definition 4.3 and compute a function $f$ of at least two inputs satisfying (E2) and (S4) of Definition 5.6. Some primary input $x$ with $d(x) \ge 2$ and some $c \in \{0, 1\}$ satisfy*

$$\Psi_2(C) - \Psi_2(N(C^{x=c})) \ge 14.$$

*Proof.* Under (S4) the function has at least three inputs, since on two bits all pair cofactors would be constants. By (E2) every one-input restriction leaves at least two essential bits and satisfies (E1). Lemma 6.12 allows us to count the decrease up to resolution. Choose an input of fan-out at least two so that no $D$-variable is fixed. By Lemma 6.11, an eliminated parity gate contributes seven units and an eliminated nonlinear gate at least three. A nonlinear gate contributes four if it plays no role or becomes constant. Each new credit contributes one more unit.

Let $T$ be the first gate in predecessor order. Its operands are distinct primary inputs $x, y$. If both feed only $T$, all dependence on the pair passes through its one output bit, giving at most two cofactors. By (S4) we may name the inputs so that $d(x) \geq 2$. Two observations will be used throughout. First, absence of an input triangle means that a successor of a gate fed by a primary input $v$ cannot itself be fed directly by $v$. Second, a gate made constant cannot be the output because the cofactor depends on at least two bits. It therefore has a successor, and every such successor is eliminated. Lemma 6.10 gives four units for each eliminated nonlinear successor of $x$ and for each eliminated nonlinear successor of $T$.

*Parity at the first gate.* If all direct successors of $x$ are parity gates, either value eliminates at least two of them, for fourteen units. Otherwise block one nonlinear successor $P$ at $x$ in the sense of Definition 4.2. Gates $T$ and $P$ contribute seven and four units. A successor of $P$ is distinct from both and contributes at least three.

*A nonlinear first gate.* Now $d(y) \geq 2$ as well. If $T$ were the only successor of $y$, blocking $T$ through $x$ would disconnect $y$, contrary to (E1) by Lemma 6.4. Write $c_T$ for the constant output of $T$ when blocked. For $v = x$ or $y$, call fixing $v$ to its blocking value for $T$ the $v$-step. If $d(v) \geq 3$, the $v$-step eliminates $T$, two other successors of $v$ and a successor of $T$, giving at least sixteen units. We may therefore assume $d(x) = d(y) = 2$. Write $U$ and $V$ for the respective successors of $x$ and $y$ other than $T$.

If $U$ is parity, the $x$-step eliminates $T, U$ and a successor of $T$, for at least $4 + 7 + 4$. The corresponding $y$-step works if $V$ is parity. If the same value of $x$ blocks both $T$ and $U$, both become constant. If they have at least two distinct successors in total, one successor of $T$ contributes at least four units, and a second gate contributes at least three more. If they have just one common successor, that gate becomes constant and has an eliminated successor, again giving at least $4 + 4 + 4 + 3$. The argument with $y, V$ applies if their blocking values at $T, V$ agree.

It remains to consider nonlinear $U, V$ with opposite blocking values at $T, U$ for $x$ and at $T, V$ for $y$. Under the $x$-step, $T$ and $U$ each contribute four. If $T$ has at least two successors their contributions bring the total to sixteen. If its unique successor $R$ is parity the total is fifteen. If $R$ is nonlinear and blocked by $c_T$, it becomes constant, bringing the total to twelve. A successor of $R$ distinct from $U$ adds at least three. If $U$ is its only successor, both operands of $U = u(x, R)$ are constant, so a successor of $U$ adds at least three. In the remaining case $R$ is nonlinear and $c_T$ does not block it. The $x$-step eliminates the three distinct gates $T, U, R$ for twelve units.

Write $c_U, c_V$ for the outputs of $U, V$ at their respective blocking values of Definition 4.2.

*Distinct second successors.* If $U = V$, all dependence on $(x, y)$ passes through the pair $(T, U)$. For $x$ at its blocking value for $T$, $T$ is constant and $U$ ranges over both values; for the other value of $x$, $U$ is constant and $T$ ranges over both values; the pair of constants $(c_T, c_U)$ is attained in both halves, so $(T, U)$ takes at most three values and $(x, y)$ has at most three distinct cofactors, contradicting (S4). Hence $U \neq V$.

*Good and bad.* Neither $U$ nor $V$ is the output of $C$: fixing $y$ to its blocking value for $V$ makes $V$ constant, so if $V$ were the output, $f|_y$ would be constant, contradicting (E1) ($f|_y$ depends on $x$); similarly for $U$. Call $V$ good if it has at least two successors, or its unique successor is parity, or its unique successor is nonlinear and blocked by the constant value $c_V$ of $V$ when blocked; otherwise $V$ is bad, with a unique nonlinear successor $V_2$ not blocked by $c_V$. Use the identical criterion with $U$ in place of $V$, writing $U_2$ for the unique nonlinear successor when $U$ is bad.

- $V$ good: fix $y$ to its blocking value for $V$ (non-blocking for $T$). Then $V$ is constant (4), $T$ is bypassed (4), and the successors of $V$ give at least $3+3$, or 7, or $4+3$ (the sole successor of $V$ becomes constant and has an eliminated successor): $\geq 14$.
- $U$ good: symmetric, $\geq 14$.
- Both bad and $V_2 \notin \{R, U\}$: the $x$-step eliminates $T, U, R$ (12). If any further gate is eliminated, it contributes at least 3, and the total is $\geq 15$. Otherwise every gate other than $T, U, R$ is a gate of $C^{x=c}$, in particular $V$ and $V_2$ ($V \notin \{T, U, R\}$ by the corner exclusion and the input-triangle condition of Definition 4.3 applied to $y$, and $V_2 \notin \{T, U, R\}$ because $T$ has primary operands and by assumption). Then no gate of $\mathrm{succ}(y) = \{T, V\}$ lies in $S_y$ and the only gate $V_2$ of $\mathrm{succ}(V)$ survives resolution, so $S_y = \{y\}$ and $S_V = \{V\}$ by Lemma 6.5(c). By Lemma 6.5(b), the only successor of $y$ in $C^{x=c}$ is the nonlinear gate $V$, and the only successor of $V$ is the nonlinear gate $V_2$. Since $d(y) = 2$ in $C$, $y$ is gained both as a $D$-variable and as a $D^*$-variable: $12 + 2 = 14$.
- Both bad and $V_2 = U$: fix $x$ to its blocking value for $U$. Then $U$ is constant (4) and $T$ is bypassed (4). The only successor of $V$ is $U$, which is constant, so $S_V = \{V\}$ by Lemma 6.5(c) if $V$ survives resolution; then $V$ has no successor in $C^{x=c}$ (Lemma 6.5(b)) and is not its output (the output gate of $C$ has no successors, so it is not $V$, and it is not in $S_V$), so $V$ is pruned. Either way $V$ is eliminated (4, role-free by Lemma 6.10(a)). Finally $U_2$ is eliminated ($\geq 3$; it is not $V$, which precedes $U$): 15.
- Both bad, $V_2 = R$ and $U_2 \notin \{R, V\}$: the $y$-step is symmetric to the first both-bad case. $T, V, R$ are eliminated and play no role (12); if a further gate is eliminated, the total is $\geq 15$; otherwise $U$ and $U_2$ are gates of $C^{y=c}$, and $x$ is gained as a $D$-variable and as a $D^*$-variable: 14.
- Both bad, $V_2 = R$ and $U_2 = V$: impossible. Put $\varepsilon$ for the blocking value of $x$ for $T$ and $\delta$ for that of $y$. Fix $x = \neg\varepsilon$, which blocks $U$. Since $U$ is bad, $c_U$ does not block $V = U_2$, so $V$ becomes a literal of $y$, equal to $c_V$ exactly when $y = \neg\delta$ (the blocking value of $y$ for $V$); and $T$ becomes a literal of $y$ equal to $c_T$ exactly when $y = \delta$. As $R$ is not blocked by $c_T$ nor by $c_V$, its blocking values are $\neg c_T$ at $T$ and $\neg c_V$ at $V$. For $y = \delta$ the $V$-operand of $R$ is $\neg c_V$ (blocking) and for $y = \neg\delta$ the $T$-operand is $\neg c_T$ (blocking). So $R$ is constant, all paths from $y$ (through $T$ and $V$, which both feed only $R$) are cut, and $f|_{x=\neg\varepsilon}$ does not depend on $y$, contradicting (E1).
- Both bad and $U_2 = R$: then $V_2 \neq R$ (else $R$ would have the three operands $T, U, V$), so either $V_2 = U$ or $V_2 \notin \{R, U\}$, the two cases already covered.

In every case the decrease is at least 14. □

# 8 The gate count for 3-SAT

We apply Lemma 7.1 to the entire circuit. When a block becomes too short to guarantee four cofactors after the next restriction, we set its output to its parent's neutral value and continue with the other blocks. Each block therefore costs $k+1$ inputs in the count of fourteen-unit steps. Full degree contributes a further three units per input, up to an additive constant, and each gate accounts for at most seven of the combined units. This is the origin of the fraction 17/7.

**Theorem 8.1** (Composition of robust blocks)**.** *Let $G$ be a composition as in Definition 5.3 whose $B$ disjoint blocks are restrictions of $k$-robust functions from Definition 5.7. Each block retains at least $k+2$ free inputs. Write $L$ for their total number. Every normalized circuit computing $G$ satisfies*

$$\Psi_2(C) \geq 14(L - (k+1)B). \tag{2}$$

*If all the blocks have full degree as in Definition 5.1, their composition satisfies*

$$C_{B_2}(G) \geq \left\lceil \frac{17L - 14(k+1)B - 3}{7} \right\rceil . \tag{3}$$

*Proof.* While every block retains at least $k+2$ bits, Lemma 5.8 verifies the hypotheses of Lemma 7.1. Restrict its selected bit to the specified value. The potential drops by at least fourteen and the free length of exactly one block decreases by one.

When a block first reaches $k+1$ bits, it is still nonconstant by Definition 5.7. If other blocks remain, assign its bits to make its value one at an AND parent or zero at an OR parent. Its sibling now replaces the parent. By Lemma 5.8, the circuit before this restriction has (E1) and the remaining composition is essential in every free bit and has (E1). Lemma 6.11 therefore gives a nonnegative decrease under resolution: even a nonlinear gate charged twice contributes $4 - 2 = 2$. By Lemma 6.12, subsequent normalization also cannot increase $\Psi_2$. When only one block remains, stop as soon as it reaches $k + 1$ bits.

Each block of initial length $\ell$ has undergone exactly $\ell - (k+1)$ one-bit steps. In the terminal circuit $D^* \leq D \leq 2A$, since each first credit uses one of the two operands of a nonlinear gate. Equation (1) gives $\Psi_2 \geq 0$. Summing the actual decreases proves inequality (2).

For the size estimate take a minimum circuit and normalize by Lemma 4.4. Every gate then belongs to one of the two essential types. Consequently $7|C| = \Psi_2 + 3A + D + D^*$. Proposition 5.5 supplies $A \geq L - 1$. Substituting this and inequality (2), and taking the ceiling of the resulting bound on the integer $|C|$, gives inequality (3). □

**Theorem 8.2** (Gate complexity of binary-encoded 3-SAT)**.** *Let $F_w$ be the decision function of Definition 2.4 for exactly $m = 2^w - 1$ clauses over $m$ available variables, with $n = 3m(w + 1)$ encoding bits. For the unrestricted binary circuit complexity of Definition 2.7, along these input lengths as $w \to \infty$,*

$$C_{B_2}(F_w) \geq \frac{17}{7}n - O\left(\frac{n \log\log n}{\log n}\right). \tag{4}$$

*More precisely, for integers $4 \leq r \leq w-1$, $3 \leq k \leq r-2$ satisfying $2^{k-2} > \log_2(6r(r-1)) + r \log_2 3$,*

$$C_{B_2}(F_w) \geq n + \left\lceil \frac{(30r - 42(k+1))(m - 2^r) - 3}{7} \right\rceil . \tag{5}$$

*An explicit restriction using the weighted modular test of Proposition 5.10 gives $C_{B_2}(F_w) \geq (17/7)n - O(n/\sqrt{\log n})$.*

*Proof.* Choose a full-degree $k$-robust test $g$ on $r$ bits using Proposition 5.9. Lemma 3.1 leaves $M = m - |g^{-1}(0)|$ positive clauses, with $B = 3M$ disjoint blocks on $L = 3Mr$ free bits. Associate each clause as two binary ORs and the conjunction of clauses as a binary AND tree. Lemma 5.8 makes its output essential in every free bit. Lemma 4.6 and inequality (3) of Theorem 8.1 now give

$$\begin{aligned} C_{B_2}(F_w) &\geq n - L + \left\lceil \frac{17L - 14(k+1)B - 3}{7} \right\rceil \\ &= n + \left\lceil \frac{(30r - 42(k+1))M - 3}{7} \right\rceil . \end{aligned} \tag{6}$$

When $30r - 42(k + 1) \geq 0$, substituting $M \geq m - 2^r$ in inequality (6) proves inequality (5). Otherwise the coefficient is a negative multiple of six, and $m - 2^r \geq 1$. The ceiling on the right of inequality (5) is then at most $-1$, so Lemmas 2.11 and 3.2 suffice.

All logarithms in the following estimates have base two. Take $r = w - \lceil \log w \rceil - 1$. Then $2^r \leq m/w$, $M = m - O(m/w)$ and $L = n - O(n \log w/w)$. The least $k$ satisfying the counting inequality of Proposition 5.9 is $\log r + O(1)$, since its right side is of order $r$. In particular $r \geq k+2$

for all sufficiently large $w$. The first expression on the right of inequality (6) is bounded below by $n + (10/7)L - 2(k+1)B - 3/7$. Since $B = O(n/w)$, the loss from $(17/7)n$ is $O(n \log w/w)$. The length formula of Definition 2.3 gives $\log n = w + O(\log w)$, proving inequality (4).

For the explicit choice use Proposition 5.10 with the same $r$. The displayed transfer in equation (6) used only robustness and full degree of the chosen block, so it applies with $k = O(\sqrt{r})$ as well. For large $w$ we again have $r \geq k + 2$. The loss is now $O(n \log w/w + nk/w) = O(n/\sqrt{\log n})$. The residue specified by Proposition 5.10 determines which variable numbers to force false in Lemma 3.1. □

# 9 Conclusion

For the clause-by-clause binary encoding specified in Section 2, deciding 3-SAT requires at least

$$\frac{17}{7}n - O\left(\frac{n \log\log n}{\log n}\right)$$

gates over the full binary basis. The lower bound allows unrestricted depth and arbitrary sharing. Its leading coefficient is approximately 2.43, compared with the elementary coefficient of one. The improvement is asymptotic and applies to the stated family of input lengths.

The central contribution is a transfer that preserves the scale of the input. By placing Boolean tests in variable identifiers, the proof retains almost the entire formula description as free input. A two-level credit measure then bounds the cost of combining those tests within one shared circuit. The resulting coefficient is a lower bound for deciding satisfiability from the original description, rather than merely a bound for an auxiliary function before an expensive reduction.

This explains the result's place in the history of circuit lower bounds. Stronger coefficients for other explicit functions do not automatically yield stronger coefficients for SAT. Conversely, a coefficient of 17/7 for SAT does not improve the numerical record across all explicit functions. The significance lies in the target function, the preservation of encoding length, and the accounting that makes the transfer work. Subject to the encoding-specific literature qualification in the introduction, the results in this research line move this SAT family beyond its elementary input-dependence bound.

The theorem remains a linear lower bound. It neither separates P from NP nor establishes an exponential lower bound matching the scale of known SAT algorithms. Those limitations leave a concrete research question: can the structure of shared variables and clauses force more than a constant amount of computation per description bit? The present proof provides a quantitative baseline and a dense embedding on which stronger arguments can be tested. Further claims must be established for the same unrestricted circuit model, with the cost of the complete input encoding accounted for.

## References


[1] C. P. Schnorr. Zwei lineare untere Schranken für die Komplexität Boolescher Funktionen. *Computing* 13:155–171, 1974.

[2] L. J. Stockmeyer. On the combinational complexity of certain symmetric Boolean functions. *Math. Systems Theory* 10:323–336, 1977.

[3] W. J. Paul. A 2.5n-lower bound on the combinational complexity of Boolean functions. *SIAM J. Comput.* 6(3):427–443, 1977.

[4] N. Blum. A Boolean function requiring 3n network size. *Theoret. Comput. Sci.* 28:337–345, 1984.

[5] M. G. Find, A. Golovnev, E. A. Hirsch and A. S. Kulikov. A better-than-3$n$ lower bound for the circuit complexity of an explicit function. *FOCS 2016*, pp. 89–98. Journal version: Improving 3$n$ circuit complexity lower bounds, *Comput. Complexity* 32(2), Paper 13, 2023.

[6] J. Li and T. Yang. $3.1n - o(n)$ circuit lower bounds for explicit functions. *STOC 2022*, pp. 1180–1193. `https://eccc.weizmann.ac.il/report/2021/023/`.

[7] O. S. Mishra and R. Santhanam. An explicit $3.5n - o(n)$ size lower bound against general Boolean circuits. ECCC TR26-260, 2026.

[8] L. Trevisan. Notes for Lecture 1, CS254, Stanford University, 7 January 2014, Section 1.4. `https://lucatrevisan.github.io/cs254-14/lecture01.pdf`.

[9] O. Lachish and R. Raz. Explicit lower bound of $4.5n - o(n)$ for Boolean circuits. *STOC 2001*, pp. 399–408.

[10] A. Kojevnikov, A. S. Kulikov and G. Yaroslavtsev. Circuit complexity of MOD-functions. PDMI Preprint 18/2008, Steklov Institute, 2008.

[11] A. Kojevnikov and A. S. Kulikov. Circuit complexity and multiplicative complexity of Boolean functions. *CiE 2010*, LNCS 6158, pp. 239–245.

[12] U. Zwick. A $4n$ lower bound on the combinational complexity of certain symmetric Boolean functions over the basis of unate dyadic Boolean functions. *SIAM J. Comput.* 20(3):499–505, 1991.

[13] K. Iwama and H. Morizumi. An explicit lower bound of $5n - o(n)$ for Boolean circuits. *MFCS 2002*, LNCS 2420, pp. 353–364.

[14] A. Golovnev, A. S. Kulikov, A. V. Smal and S. Tamaki. Circuit size lower bounds and #SAT upper bounds through a general framework. *MFCS 2016*, LIPIcs 58, 45:1–45:16. Journal version: Gate elimination: circuit size lower bounds and #SAT upper bounds, *Theoret. Comput. Sci.* 719:46–63, 2018.

[15] C. P. Schnorr. The multiplicative complexity of Boolean functions. *AAECC-6*, LNCS 357, pp. 45–58, 1989.

[16] L. Trevisan. Notes for Lecture 3, CS254, Stanford University, 2014. `https://theory.stanford.edu/~trevisan/cs254-14/lecture03.pdf`.

[17] C. Carlet. *Boolean Functions for Cryptography and Coding Theory.* Cambridge University Press, 2021.

[18] E. Kushilevitz and N. Nisan. *Communication Complexity*. Cambridge University Press, 1997.

[19] É. Balandraud. An addition theorem and maximal zero-sum free sets in $\mathbb{Z}/p\mathbb{Z}$. *Israel J. Math.* 188:405–429, 2012. arXiv:0907.3492.

[20] P. L. Tchebychev. Mémoire sur les nombres premiers. *J. Math. Pures Appl.* 17:366–390, 1852.

[21] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009, Chapter 14. https://doi.org/10.1017/CBO9780511804090.017.

[22] L. Fortnow. When will we show P $\neq$ NP? *Computational Complexity*, 6 January 2003. https://blog.computationalcomplexity.org/2003/01/when-will-we-show-p-np.html.

[23] U. Schöning. A probabilistic algorithm for $k$-SAT and constraint satisfaction problems. *FOCS 1999*, pp. 410–414. https://doi.org/10.1109/SFFCS.1999.814612.

[24] T. Hertli. 3-SAT faster and simpler—Unique-SAT bounds for PPSZ hold in general. *FOCS 2011*, pp. 277–284. https://arxiv.org/abs/1103.2165.

[25] Companion research manuscripts. *Circuit Lower Bounds: $4n - o(n)$ for an Explicit Function in P and $7n/3 - o(n)$ for Binary-Encoded 3-SAT*, 29 October 2026; *Fan-Out Credits in Weighted Gate Elimination: $(12/5 - o(1))n$ and $(17/7 - o(1))n$ Lower Bounds for Binary-Encoded 3-SAT*, 30 October 2026. P_vs_NP repository, working manuscripts.

[26] O. Goldreich. Improving the lower bound on size complexity of explicit functions and some limitations on subsequent improvements. Oded's Choices, no. 202, 2016. https://www.wisdom.weizmann.ac.il/~oded/MC/202.html.

[27] Princeton University, Department of Computer Science. Sanjeev Arora: faculty profile. https://www.cs.princeton.edu/people/profile/arora.

[28] Harvard University, Kempner Institute. Boaz Barak: faculty profile. https://kempnerinstitute.harvard.edu/people/our-people/boaz-barak/.

[29] University of Chicago, Leadership and Society Initiative. Lance Fortnow: biography. https://leadforsociety.uchicago.edu/bio/lance-fortnow/.

[30] Weizmann Institute of Science. Israel Prize awarded to Oded Goldreich. 4 August 2022. Weizmann Compass, institutional announcement.